Nhill 2.0 — Developer guide

Français : ../fr/guide-developpement.md

How to understand the code, change it without breaking its guarantees, and have the CI check it. Detailed architecture and business rules: `docs/technique.md`; API: `docs/api.md` (both reference documents are in French).

1. Architecture at a glance

```
Nhill Business          Nhill POS              Nhill Console
(Windows/Linux/Android) (Windows/Linux)        (Windows/Linux, creators)
  | HTTPS, short tokens   | HTTPS, short tokens    | HTTPS, admin token (password + TOTP)
  ▼                       ▼                        ▼
┌──────────────── TLS proxy (Caddy) ──────────────────────────────────┐
│                  nhill-server (C++17, cpp-httplib)                  │
│  http/app (routing, rights, idempotency, errors) — services/*       │
│  setup (setup mode) — providers — jobs — pdf                        │
└──────────────┬──────────────────────────────────────────────────────┘
               | role nhill_app (owns nothing, forced RLS)
               ▼
  PostgreSQL 15+ (Supabase or self-hosted) — schema "nhill", db/migrations
```

Invariants never to break:

- the client holds **no database credentials** and only talks to the API;
- each request is a transaction; `Tx::setTenant` sets `app.establishment_id` and the database itself filters business tables (`FORCE ROW LEVEL SECURITY` on 30 tables);
- amounts are integers in minor units (`common/`: `Money`), never floating point; currencies are never added together;
- an issued document is immutable; corrections go through a credit note;
- the activity log is append-only (no route changes it);
- sensitive write operations are idempotent (`Idempotency-Key`).

2. Repository layout

Folder	Content
`common/`	C++ library shared by server and clients: amounts, establishment codes, validation (including the password rule), version
`server/src/`	`http/` (application, routing), `services/` (domains), `setup.*`, `jobs/`, `pdf/`, `migrate.*`, `db.*`, `config.*`
`server/profiles/`, `server/i18n/`	country tax profiles, server FR/EN labels (PDF, activity log)
`db/migrations/`	`0001` … `0010`, digests checked at start-up
`client/src/`	`core/` (ApiClient, Session, Validator, AppContext), `business/`, `console/`, `pos/`
`client/qml/Nhill/`	`Ui/` (design system), `Business/`, `Console/`, `Pos/`
`client/i18n/`	`nhill_fr.ts`, `nhill_en.ts`

Folder	Content
`tests/`	`common/`, `server/` (throwaway PostgreSQL), `client/`, `support/minigtest/`
`packaging/`	Debian (`linux/`), Windows (NSIS, MinGW), Android
`scripts/`	build, verification, release, QA, i18n, development
`website/`	marketing website and form backend (independent from the application)
`docs/`, `legal/`, `marketing/`	FR/EN documentation (Markdown = source, generated PDFs)

3. Development machine

```
scripts/install-deps-linux.sh --with-packaging --with-docs --yes   # Debian 13 / Ubuntu 24.04
scripts/build-linux.sh                                             # build + tests
NH_DEV_STATE=/tmp/nhdev scripts/dev/dev-stack.sh start build       # throwaway PostgreSQL +
API 127.0.0.1:8080
python3 scripts/dev/seed-demo.py --email demo@example.cd           # DEMO data
NHILL_SERVER=http://127.0.0.1:8080 build/client/nhill-business
NH_DEV_STATE=/tmp/nhdev scripts/dev/dev-stack.sh stop
```

- `dev-stack.sh` finds PostgreSQL in `/usr/lib/postgresql/<version>/bin` (newest); `NH_PG_BIN` forces another folder.
- Local administration console: `build/server/nhill-server --config /tmp/nhdev/server.env admin-bootstrap admin@example.dev "Admin"`, then “First access” in `build/client/nhill-console`.
- Demo data is all marked “(démo)”; the script refuses a non-local server.
- **C++17 is mandatory**: the distribution’s libpqxx and cpp-httplib are built as C++17; building Nhill as C++20 breaks the ABI.

4. Tests

Suite	Command	Count (2.0.0)	Dependencies
`common`	`ctest -R common` or `scripts/test-common-standalone.sh`	20	none (the standalone script uses `tests/support/minigtest`)
`server`	`ctest -R server`	84	local PostgreSQL 15+ (`initdb`, `pg_ctl`)
`setup`	`ctest -R setup`	6	PostgreSQL
`client`	`ctest -R client`	13	Qt 6
Website backend	`cd website/backend && python3 -m unittest discover -s tests`	13	Python 3 (SQLite); PostgreSQL in CI
QML (static)	`python3 scripts/qa/qml-check.py`	95 files	Node.js
Website (static)	`python3 website/tools/check_site.py website`	24 pages	Python 3

Server tests start a throwaway PostgreSQL; as root they use the `postgres` user. A test that depends on rate limits calls `resetIpThrottles()` first.

Rule: any change to a business or security rule comes with a test that fails before the change and passes after it.

5. User interface (QML): design system

Components of `Nhill.Ui`, to be used instead of bare Qt controls:

Component	Use
`Theme`	tokens: colours (light/dark, WCAG AA contrast checked), spacing `s1…s6`, sizes `fs_xs` (12) … `fs_xl`, radii, `fitWidth()` / `fitHeight()` to size a window for the screen
`FbButton`	`primary`, `secondary`, `ghost`, `danger` variants; `tip` = tooltip (mandatory for an icon-only button); focus ring
`FbTextField`	`echoMode: TextInput.Password` adds the show/hide button; `accessibleName`
`Field`	label + help + error (`error`); gives the field its accessible name
`DataTable`	columns with `minWidth`, horizontal scrolling, tooltips, keyboard navigation
`Notifier`	messages (`Nav.toast`); errors 8 s, 4 at most, `AlertMessage` role
`Icon`, `Banner`, `FbCard`, `FbComboBox`, `FbSwitch`, `DateField`, `TabStrip`	see the files

Rules: no hard-coded colour or size in pages (theme tokens); every visible text goes through `qsTr()`; every window uses `Theme.fitWidth/fitHeight` and a minimum bounded by the screen; test at 1366×768, 1280×720, 390×844 (phone) and in dark theme.

Static check: `python3 scripts/qa/qml-check.py` (JavaScript syntax of handlers, files listed in the CMakeLists, untranslated texts). Headless screenshots: `scripts/qa/ui-screenshots.sh` (Xvfb).

6. Translations

Sources are in English (`qsTr("…")`). After adding texts:

```
lupdate client -ts client/i18n/nhill_fr.ts client/i18n/nhill_en.ts   # Qt tool
python3 scripts/i18n/ts-set.py translations.json                     # fills the FR
translations from a JSON file
```

Server labels (PDF, activity log): `server/i18n/fr.json` and `en.json`; every `action.*` key must exist in both.

7. Database and migrations

- New migration = new file `db/migrations/00NN_name.sql`. **Never edit a published migration**: the server compares SHA-256 digests and refuses to start.
- Any new business table: `establishment_id` column, `ENABLE` + `FORCE ROW LEVEL SECURITY`, policy `USING (establishment_id = (SELECT nh_current_establishment()))` (pattern: `0009_pos.sql`), minimal grants for `nhill_app`, foreign-key indexes; add an isolation test.
- Setup mode checks isolation with 10 controls: add one if the new table holds sensitive data.

8. API

- Prefix `/api/v1`; an incompatible change requires `/api/v2` (with `v1` served during the transition). `GET /api/v1/meta` publishes `min_client_version`.
- Errors: `{"error": {"code": …, "message": …, "request_id": …}}` (plus `field` for rejected input); codes are translated on the client (`AppContext`). Add the FR/EN message of any new code.
- Inputs are parsed strictly (`services/common`), sizes bounded; SQL is never built by concatenation (`$1…` parameters).

9. Security in the code

- Passwords: `nhill::validation::checkPassword` (8 characters minimum, common-password list) on the client **and** the server; Argon2id hashing (`crypto::hashPassword`).
- Constant-time comparison of secrets (`crypto::constantTimeEquals`).
- Provider secrets sealed with a sub-key of `NH_SECRET_KEY`.
- Nothing personal in logs; client errors masked on the server.
- See security.md and the audit 03-security-audit.md.

10. Continuous integration

Workflow	Content
`ci.yml`	build, tests (common, server, setup, client), QML check, screenshots
`linux-packages.yml`	`.deb` for Ubuntu 24.04 and Debian 13 in clean containers + test installation
`windows.yml`	Qt 6.5.3 MSVC, NSIS installers, silent test installation
`android.yml`	SDK/NDK, Qt for Android, APK
`docs-website.yml`	up-to-date PDFs, marketing website, backend tests (SQLite and PostgreSQL)

CI artefacts are the ones to publish; never publish a binary built on an untracked personal machine.

11. Version

Single source: `project(nhill VERSION x.y.z)` in `CMakeLists.txt`. Scripts, packages, the Windows installer and the APK read it. Also update: `common/include/nhill/version.hpp` (`kProductVersion`), `QT_ANDROID_VERSION_CODE`, Windows `.rc` resources, FR/EN changelogs.